Покажи обработку ошибок в асинхронных агентах

Обработку ошибок лучше делать **на двух уровнях**: внутри агента — чтобы сбой не ломал весь рой, и в координаторе — чтобы собрать все ошибки и решить, что делать. Для параллельного запуска используйте `asyncio.gather(..., return_exceptions=True)` или `asyncio.TaskGroup`; второй надёжнее, потому что при ошибке отменяет остальные задачи. [1] [2] [3]

1. Ошибки внутри агента

Добавьте в `BaseAgent` безопасную обёртку:

```
# src/les/agents/base_agent.py
from abc import ABC, abstractmethod
from typing import Dict, Any
import logging

logger = logging.getLogger(__name__)

class BaseAgent(ABC):
    def __init__(self, name: str, threshold: float = 0.3):
        self.name = name
        self.threshold = threshold

    @abstractmethod
    async def evaluate(self, state: Dict[str, float]) -> Dict[str, Any]:
        ...

    @abstractmethod
    async def act(self, state: Dict[str, float]) -> Dict[str, Any]:
        ...

    async def safe_act(self, state: Dict[str, float]) -> Dict[str, Any]:
        try:
            return await self.act(state)
        except asyncio.CancelledError:
            raise
        except Exception as exc:
            logger.exception("Agent %s failed", self.name)
            return {
                "agent": self.name,
                "action": "error",
                "reason": f"{type(exc).__name__}: {exc}",
```

```
                "recoverable": True,
            }
```

Важно: `asyncio.CancelledError` нужно **пробрасывать дальше**, а не превращать в обычную ошибку агента — иначе нельзя корректно остановить симуляцию. [4] [5]

2. Координатор с `gather`

```
# src/les/swarm/swarm_manager.py
import asyncio
import logging
from typing import Dict, List, Any

logger = logging.getLogger(__name__)

class SwarmManager:
    def __init__(self, agents):
        self.agents = agents
        self.log: List[dict] = []

    async def run(self, state: Dict[str, float]) -> Dict[str, Any]:
        tasks = [agent.safe_act(state) for agent in self.agents]

        results = await asyncio.gather(*tasks, return_exceptions=True)

        clean_results: List[dict] = []
        errors: List[dict] = []

        for agent, result in zip(self.agents, results):
            if isinstance(result, BaseException):
                errors.append({
                    "agent": agent.name,
                    "error": f"{type(result).__name__}: {result}",
                })
                logger.error("Agent %s failed: %s", agent.name, result)
            else:
                clean_results.append(result)
                self.log.append(result)

        conflicts = [
            r for r in clean_results
            if r.get("action") in {"escalate", "block"}
        ]

        decision = self.decide(clean_results, conflicts, errors)

        return {
            "state": state,
            "results": clean_results,
            "conflicts": conflicts,
            "errors": errors,
            "decision": decision,
        }

    def decide(
```

```
        self,
        results: List[dict],
        conflicts: List[dict],
        errors: List[dict],
    ) -> str:
        if any(r.get("action") == "block" for r in results):
            return "STOP"

        if errors:
            return "ESCALATE"

        if conflicts:
            return "ESCALATE"

        return "CONTINUE"
```

`return_exceptions=True` не позволяет одной ошибке прервать ожидание остальных агентов: все результаты, включая исключения, попадают в итоговый список.[2] [6] [1]

3. Более строгий вариант: `TaskGroup`

Если ошибка агента должна **останавливать весь рой**, используйте `TaskGroup`:

```
async def run_strict(self, state: Dict[str, float]) -> Dict[str, Any]:
    results = []

    async with asyncio.TaskGroup() as tg:
        tasks = [
            tg.create_task(agent.safe_act(state))
            for agent in self.agents
        ]

    results = [task.result() for task in tasks]
    return {
        "state": state,
        "results": results,
        "decision": self.decide(results, [], []),
    }
```

При исключении в одной задаче `TaskGroup` отменяет остальные задачи и собирает исключения в `ExceptionGroup`; это даёт более сильные гарантии согласованности, чем `gather`.[3] [5] [1]

4. Пример запуска

```
import asyncio
from les.swarm import SwarmManager
from les.agents import (
    SurvivalAgent, OthersAgent, MeaningAgent,
    ProcedureAgent, AdaptationAgent, DiversityAgent
)
```

```
async def main():
    manager = SwarmManager([
        SurvivalAgent("survival"),
        OthersAgent("others"),
        MeaningAgent("meaning"),
        ProcedureAgent("procedure"),
        AdaptationAgent("adaptation"),
        DiversityAgent("diversity"),
    ])

    state = {
        "S": 0.2, "O": 0.8, "M": 0.8,
        "P": 0.8, "A": 0.8, "D": 0.8,
    }

    result = await manager.run(state)

    print("Decision:", result["decision"])
    print("Errors:", result["errors"])
    print("Conflicts:", result["conflicts"])

asyncio.run(main())
```

5. Политика ошибок

Ситуация	Действие координатора
Агент вернул `continue`	Продолжить
Агент вернул `escalate`	Эскалация
Агент вернул `block`	Полная остановка
Агент выбросил исключение	Эскалация, но остальные агенты завершаются
Получен `CancelledError`	Корректная отмена, без подмены на обычную ошибку

Такой дизайн сохраняет главный принцип роя: **отказ одного агента не должен автоматически разрушать всю систему**, но должен быть видимым и влиять на решение координатора.[7] [2] [3]

⁂

1. https://docs.python.org/ja/3/library/asyncio-task.html?highlight=taskgroup
2. https://docs.dbos.dev/python/tutorials/workflow-tutorial
3. https://github.com/python/cpython/blob/22762c0d/Doc/library/asyncio-task.rst
4. https://docs.python.org/3/library/asyncio-exceptions.html
5. https://raw.githubusercontent.com/python/cpython/main/Lib/asyncio/taskgroups.py
6. https://docs-python.ru/standart-library/modul-asyncio-python/funktsija-gather-modulja-asyncio/
7. https://docs.python.org/3/tutorial/errors.html
8. https://github.com/python/cpython/blob/22762c0d/Lib/asyncio/taskgroups.py

9. https://docs.python.org/uk/3/library/asyncio-task.html
10. https://docs.python.org/3/library/asyncio-dev.html
11. https://docs.python.org/3/library/asyncio-task.html?highlight=gather
12. https://docs.python.org/3/library/asyncio-api-index.html
13. https://docs.python.org/3/library/asyncio-dev.html?highlight=exception